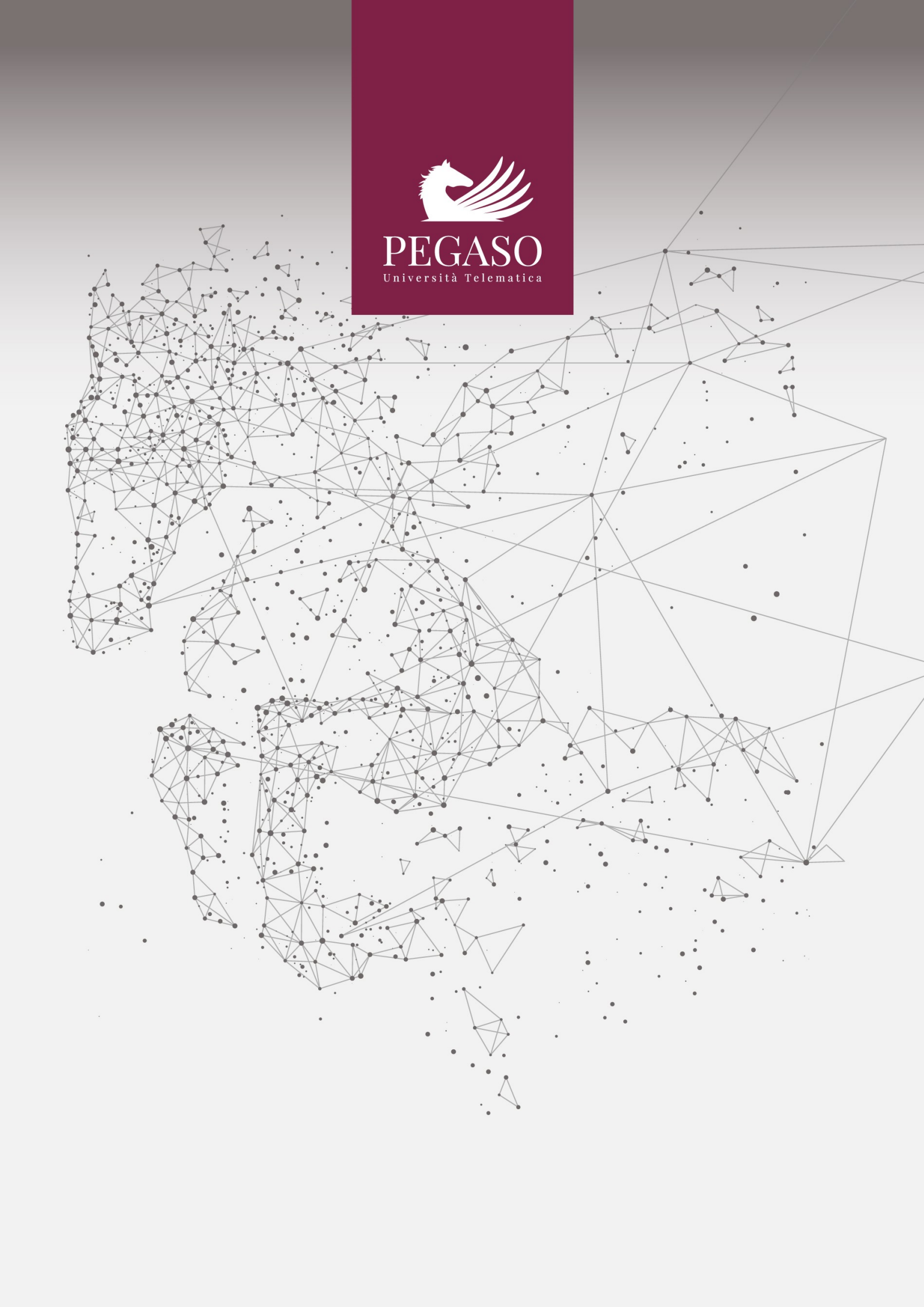

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. COOKIES ..... 3**
**2. STRUTTURA DI UN COOKIE ..... 6**
**3. IMPLEMENTAZIONE ..... 9**
**BIBLIOGRAFIA E SITOGRAFIA ..... 13**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. Cookies

HTTP è stato originariamente progettato per essere un protocollo anonimo, **stateless** di tipo request/response: ciascuna richiesta effettuata, anche dallo stesso client, è indipendente da quelle effettuate in precedenza. Il protocollo base prevede addirittura di chiudere la connessione TCP ad ogni richiesta.

Un server senza stati (**Stateless Server**) è un server che tratta ogni richiesta come una transazione indipendente scorrelata da ogni precedente richiesta. Un server con stati (**Stateful Server**) ha una tabella che permette di registrare in modo incrementale le informazioni e lo stato delle interazioni in corso con i client, incluse le risposte ad ogni richiesta, in modo che il server possa tener traccia di tutte le richieste effettuate dal client.

Per ovviare a questi problemi sono stati introdotti i cookies, che rappresentano oggi un modo consolidato per identificare gli utenti e permettere sessioni persistenti.

I cookie sono stringhe di dati di piccole dimensioni che vengono memorizzate direttamente nel browser e che fanno parte del protocollo HTTP. I cookie vengono generalmente impostati da un server Web utilizzando la risposta **Set-Cookie HTTP-header**. Quindi, il browser li aggiunge automaticamente a (quasi) tutte le richieste allo stesso dominio utilizzando l'intestazione HTTP Cookie.

Un cookie è costituito dai seguenti componenti:

- Nome.
- Valore.
- Zero o più attributi (coppie nome/valore). Gli attributi memorizzano informazioni come la scadenza del cookie, il dominio e i flag (come Secure e HttpOnly).

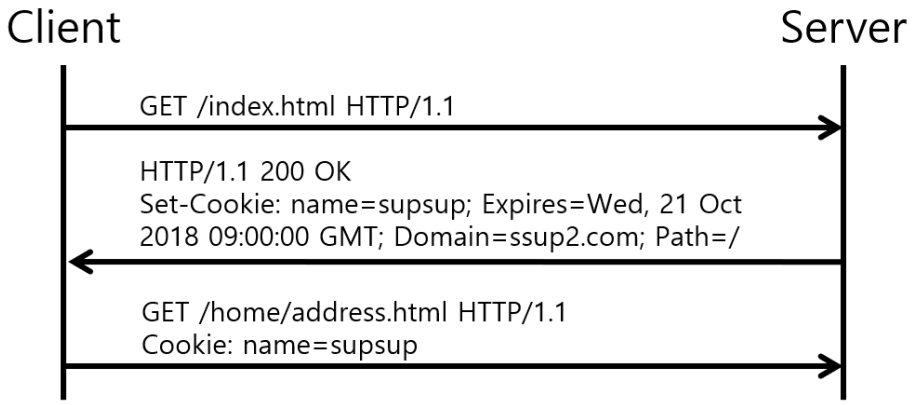



***Figura 1** - Esempio di Set-Cookie*

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

**Attenzione**: i browser non includono gli attributi dei cookie nelle richieste al server: inviano solo il nome e il valore del cookie; gli attributi dei cookie vengono utilizzati dai browser per determinare quando eliminare un cookie, bloccare un cookie o se inviare un cookie al server.

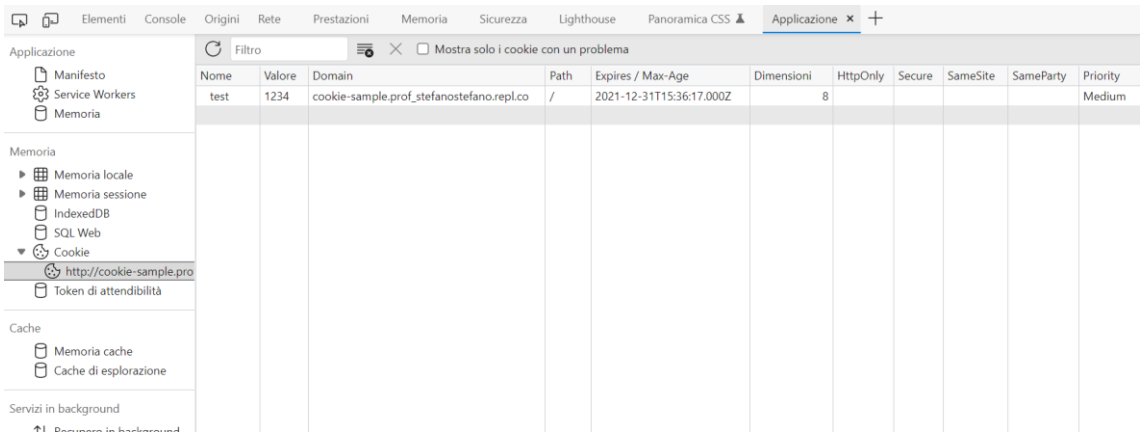



***Figura 2*** *- Cookie Inspect*

Un classico utilizzo dei cookie è quello per fornire agli utenti un modo per registrare gli articoli che desiderano acquistare mentre navigano in un sito Web (un "carrello della spesa" o "carrello della spesa" virtuale).

Oggi, tuttavia, il contenuto del carrello degli acquisti di un utente viene solitamente archiviato in un database sul server, piuttosto che in un cookie sul client. Per tenere traccia di quale utente è assegnato a quale carrello, il server invia un cookie al client che contiene un identificatore di sessione univoco (in genere, una lunga stringa di lettere e numeri casuali). Poiché i cookie vengono inviati al server con ogni richiesta effettuata dal client, l'identificatore di sessione verrà rispedito al server ogni volta che l'utente visita una nuova pagina del sito Web, il che consente al server di sapere quale carrello della spesa mostrare all'utente.

I cookie possono essere utilizzati per ricordare informazioni sull'utente al fine di mostrare contenuti pertinenti a quell'utente nel tempo. Molti siti Web utilizzano i cookie per la personalizzazione in base alle preferenze dell'utente. Gli utenti selezionano le loro preferenze inserendole in un modulo web e inviando il modulo al server. Il server codifica le preferenze in un cookie e invia il cookie al browser. In questo modo, ogni volta che l'utente accede ad una pagina del sito, il server può personalizzare la pagina in base alle preferenze dell'utente.

Se l'utente richiede una pagina del sito, ma la richiesta non contiene cookie, il server presume che questa sia la prima pagina visitata dall'utente: il server crea un identificatore univoco (tipicamente una stringa di lettere e numeri casuali) e lo invia come cookie al browser insieme alla pagina richiesta. Da questo momento in poi, il cookie verrà automaticamente inviato dal browser al server ogni volta che viene

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

richiesta una nuova pagina dal sito. Il server non solo invia la pagina come di consueto, ma memorizza anche l'URL della pagina richiesta, la data/ora della richiesta e il cookie in un file di registro.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. Struttura di un cookie

Vi sono diverse tipologie di cookies:

- **Session cookie** (o transient o in-memory cookie): il loro span temporale è limitato al ciclo di vita del browser. Quando l'utente chiude il browser vengono automaticamente eliminati.
- **Persistent cookie**: il loro span temporale è definito in modo esplicito dal server. Pertanto, restano validi anche dopo la chiusura del browser o lo spegnimento della macchina client.
- **Secure cookie**: un cookie "secure" può essere trasmesso dal client soltanto se si utilizza HTTPS. Questo per evitare che i dati contenuti nel cookie possano essere rubati spiando il traffico sulla rete.
- **HttpOnly cookie**: cookie che non possono essere letti da API client-side come.
- Javascript. Questo evita il furto dei cookie attraverso cross-site scripting.
- **SameSite cookie**: introdotti da Chrome, sono cookie che possono essere inviati soltanto se il server A ha anche fornito la risorsa (pagina HTML) contenente il link ad una risorsa contenuta in A. Questo evita che un sito di un server B possa contenere un link verso A per scatenare azioni malevole sfruttando l'autenticazione esistente del client (cross-site request forgery).

Analizziamo il formato di HTTP request header ed HTTP response header relativamente al cookie:

**HTTP request header**

```
Cookie: name1 = value1 [; name2 = value2 ] ...
```

**HTTP response header**

```
Set-Cookie: name=value [; expires=date] [; max-age=number] [;
path=path_value] [; domain=domain_value] [; secure] [; HttpOnly]
[; SameSite]
```

Le parentesi quadre indicano i valori "opzionali".

Analizziamo nel dettaglio i vari campi presenti nel Set-Cookie:

- **name = value**: è la coppia chiave-valore da memorizzare sul client, ad esempio sessionid=12345.
- **expires = date**: è la data di scadenza del cookie; dopo tale data viene automaticamente eliminato dal client. Se l'attributo non è presente, il cookie è considerato di sessione e viene rimosso alla chiusura del browser.

PEGASO Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

- **max-age = number**: indica il numero di secondi fino alla scadenza del cookie; un numero zero o negativo farà scadere immediatamente il cookie.
- **path = path_value**: con l'attributo path il cookie viene legato ad una particolare risorsa; soltanto quando il client richiede una risorsa il cui path corrisponde al path specificato è tenuto ad inviare il contenuto del cookie.
- **domain = domain_value**: con l'attributo domain il cookie viene legato ad un particolare dominio; per ragioni di sicurezza, un server può specificare soltanto domini o sotto-domini del proprio dominio; se ad esempio il server opera sul dominio foo.com:
    - se l'attributo domain non è presente, il cookie verrà inviato anche a risorse richieste a sub.foo.com
    - se l'attributo domain=foo.com, il cookie sarà inviato soltanto per risorse richieste a foo.com
    - In nessun caso al server è permesso impostare cookie su domini diversi da foo.com, ex. faa.com; se fosse possibile, un server potrebbe "sovrascrivere" cookie di altri soggetti
- **secure**: se l'attributo è impostato, il client può inviare il cookie soltanto se la connessione è di tipo HTTPS.
- **HttpOnly**: Se l'attributo è impostato, il client non può dare accesso alle informazioni del cookie alle API lato client come Javascript, flash, etc.
- **SameSite**: l'attributo rende il cookie di tipo "SameSite" per impedire attacchi di cross-site request forgery (non supportato da tutti i browser).

Da precisare che nel caso in cui **expires** e **max-age** siano valorizzati, **max-age** ha la precedenza.

Riportiamo di seguito alcuni esempi di cookies:

- Session cookie:
    - Set-Cookie: sessionId=1234
    - Tale cookie non ha valorizzati expires o max-age e dunque è di sessione
- Permanent cookie:
    - Set-Cookie: id=1234; Expires=Wed, 21 Oct 2025 11:30:00 GMT
    - Tale cookie ha valorizzato il campo expires pertanto è permanente
- Permanent cookie:
    - Set-Cookie: id=1234; Max-Age=2001000
    - Tale cookie ha valorizzato il campo max-age pertanto è permanente
- Non valido:
    - Set-Cookie: id=1234; Domain=mydomain.it

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

  - Se tale cookie fosse rilasciato da un server ospitato su un dominio del tipo abc.it, questo sarebbe considerato "non valido" perché il campo domain contiene un valore diverso dal dominio di provenienza

- Non valido:
  - Set-Cookie: id=1234; Domain=sub.mydomain.it
  - Se tale cookie fosse rilasciato dal dominio mydomain.it, questo sarebbe considerato "non valido" perché il campo domain contiene un valore del sottodominio

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. Implementazione

Consideriamo di seguito il codice in Javascript per il settaggio dei cookies:

```
function setSessionCookie(name, value) {
	var cookie = encodeURIComponent(name) + "=" + encodeURIComponent(value);
	document.cookie = cookie;
}
function setPersistentCookie(name, value, days) {
	let date = new Date(Date.now() + days*86400e3);
	date = date.toUTCString();
	var cookie=encodeURIComponent(name) + "=" + encodeURIComponent(value) +
";expires=" + date + ";
	document.cookie = cookie;
}
```

Sono state definite due funzione:

- **setSessionCookie** determina il settaggio di un cookie di sessione (senza expires o max-age)
- **setPersistentCookie** determina il settaggio di un cookie persistente (con expires valorizzato)

La coppia name / value in ingresso determina rispettivamente:

- nome del cookie;
- valore del cookie.

Al fine di gestire correttamente il settaggio del cookie, tali valori sono "encoded" tramite la funzione **encodeURIComponent** ed infine il cookie è inviato dal server verso il client tramite l'istruzione: **document.cookie=cookie**

Una generica "setCustomCookie" può essere implementata nel seguente modo:

```
// example: setCookie('myName', 'myValue', {secure: true, 'max-age': 3600});
function setCustomCookie(name, value, options = {}) {
  if (options.expires instanceof Date) {
    options.expires = options.expires.toUTCString();
  }
  let cookie = encodeURIComponent(name) + "=" + encodeURIComponent(value);
  for (let optionKey in options) {
    cookie += "; " + optionKey;
    let optionValue = options[optionKey];
    if (optionValue !== true) {
```

PEGASO Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

```
            cookie += "=" + optionValue;
          }
        }
        document.cookie = cookie;
        console.log("Sucessfully created custom cookie:\n" + cookie);
      }
```

Nel momento in cui un client si presenta al server è necessario da parte del server verificare se un cookie è presente; in tal caso il codice in Javascript per il recupero del cookie è il seguente:

```
function getCookie(cname) {
      let name = cname + "=";
      let decodedCookie = decodeURIComponent(document.cookie);
      let ca = decodedCookie.split(';');
      for(let i = 0; i <ca.length; i++) {
            let c = ca[i];
            while (c.charAt(0) == ' ') {
                  c = c.substring(1);
            }
            if (c.indexOf(name) == 0) {
                  return c.substring(name.length, c.length);
            }
      }
      return "";
}
```

Tale funzione "recupera" un cookie di nome cname; se il cookie è presente, esegue lo split dei parametri mediante l'istruzione **decodedCookie.split(';')** e successivamente restituisce il valore per tale cookie.

Se si vuole "cancellare" un cookie, l'opzione da seguire è quella di settare una data di expires nel passato; di seguito il codice in Javascript che esegue tale operazione:

```
function deleteCookie(name) {
      var expires = -1
      let date = new Date(Date.now() + expires*86400e3);
      date = date.toUTCString();
      var expires = "expires="+ date;
      document.cookie = name + "=" + 0 + ";" + expires + ";path=/";
}
```

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Ipotizzando che le funzioni Javascript illustrate sia presenti in un file cookies.js, il codice HTML associato al setup del cookie è il seguente:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width">
    <title>Main Content</title>
    <link href="style.css" rel="stylesheet" type="text/css" />
    <script src="cookie.js"></script>
  </head>
  <body>
    Hello Main Content
    <script>
      setPersistentCookie("main",1234,10)
    </script>
  </body>
</html>
```

Supponiamo che all'interno del nostro server vi sia la seguente struttura:

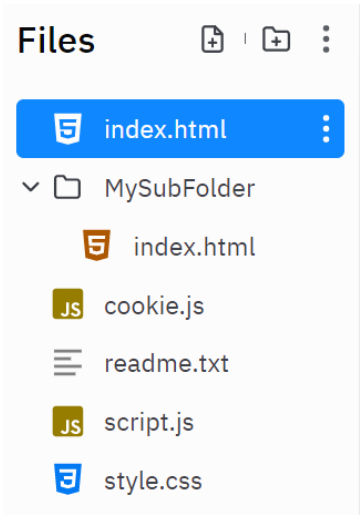



***Figura 3*** *- Struttura del web site*

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Immaginiamo che vi siano 2 files index.html in 2 percorsi differenti:

- nella main root;
- nella sottocartella MySubFolder.

Supponiamo inoltre che ognuna di queste pagine contenga un Javascript che esegua i seguenti set-cookie:

  - l'index.html nella root esegua un set-cookie "main" che sarà disponibile nel path / → setPersistentCookie("main",1234,10)
  - l'index.html in MySubFolder esegua un set-cookie "sub" che sarà disponibile nel path /MySubFolder → setCustomCookie("sub",5678,{'path':'/MySubFolder','max-age':1})

Il risultato di questo settaggio sarà che:

- nell'index.html della root, sarà disponibile SOLO il main cookie;
- nell'index.html della MySubFolder saranno disponibili entrambi i cookies.

In altre parole:

- il main cookie sarà disponibile su tutti i paths del sito;
- il sub cookie sarà disponibile solo nella MySubFolder.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Bibliografia e sitografia


- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- F. Bergamasco - WEB APPLICATIONS AND TECHNOLOGIES - CT0142
- Significato e differenza tra Stateless e Stateful in informatica | Informatica e Ingegneria Online (vitolavecchia.altervista.org)


PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*